Modelador CSG

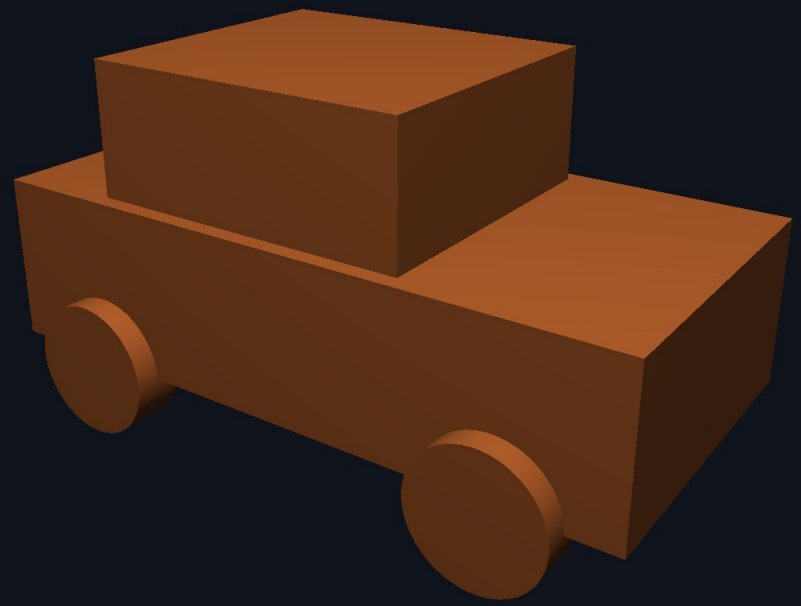
TIAGO MELO JUCÁ

MARCOS VINICIUS PEREIRA ARAUJO

LUIS HENRIQUE MOURA DA SILVA

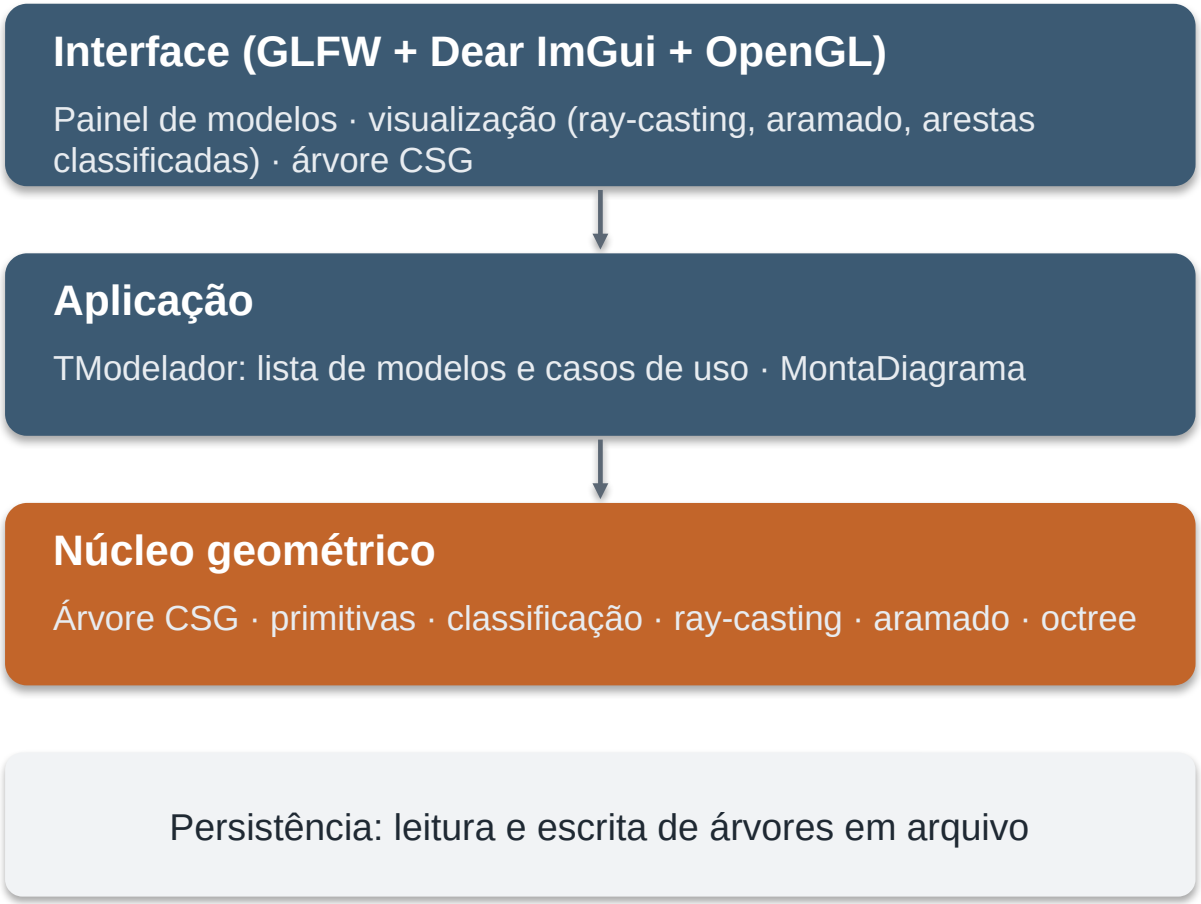
FELIPE GABRIEL LIMA FLORINDO

JOAO ITALO DE ALMEIDA CAMPOS



Arquitetura

Camadas, casos de uso e dependências



Casos de uso (TModelador)

- Criar bloco, esfera e cilindro
- Unir, intersectar, transladar, rotacionar e escalar
- Calcular volume, salvar e carregar
- Cada operação cria um modelo novo, com identificador que não muda; as entradas continuam na lista

Regras de dependência

- O núcleo não depende de janela, OpenGL ou Dear ImGui
- A interface só coleta parâmetros e chama os casos de uso
- C++17, CMake; GLFW, Dear ImGui e GoogleTest baixados pelo CMake

Árvore CSG

Gramática e tipos de nó

```
<CSG tree> ::= <primitive>
  | <CSG tree> <set op> <CSG tree>
  | <CSG tree> <rigid motion>
```

Primitiva: sempre folha; guarda o tipo e os parâmetros de dimensão

Operação booleana: dois filhos ordenados, esquerda e direita; a ordem importa na diferença

Movimento rígido: translação ou rotação, com três valores; um único filho

Códigos: os valores dos enums são os códigos do arquivo: B, E, C; U, I, D; T, R

```
enum class ETipoNoCSG {
  PRIMITIVA,
  OPERACAO_BOOLEANA,
  MOVIMENTO_RIGIDO
};

class TNoCSG {
  ETipoNoCSG tipo;
  shared_ptr<const TPrimitiva> primitiva;
  EOperacaoBooleana operacao;  // U, I, D
  TMovimentoRigido movimento;  // T, R
  TArvoreCSG esquerda;         // ou filho
  TArvoreCSG direita;
};
```

Os acessores conferem o tipo: Filho() de uma união lança logic_error

Árvore CSG

Modelagem local

Duas opções de modelagem

- Global: a matriz de transformação fica guardada na primitiva
- Local: a transformação é um nó da árvore, acima da primitiva (a adotada)

Consequências da modelagem local

- Toda primitiva fica em posição canônica: classificar contra ela usa só as equações canônicas
- Todo algoritmo recursivo trata três casos; no movimento, leva a entidade ao sistema do filho
- Mover uma subárvore inteira é acrescentar um nó acima dela
- Posição canônica: bloco e esfera centrados na origem; cilindro com eixo z e base em z = 0

```
Linha do arquivo:
  No1; C; S 0.4,0.3; R 90,0,0; T 1,0,0

Árvore criada:
  T(1, 0, 0)
  └─ R(90°, 0, 0)
     └─ Cilindro(r = 0.4, h = 0.3)

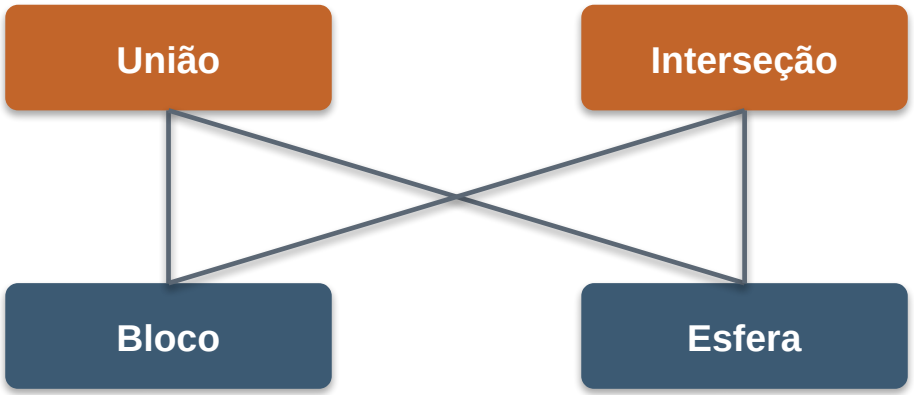
Primeiro gira, depois translada:
o nó de cima é o último aplicado.
```

Árvore CSG

Nós imutáveis e compartilhados

```
using TArvoreCSG = shared_ptr<const TNoCSG>;

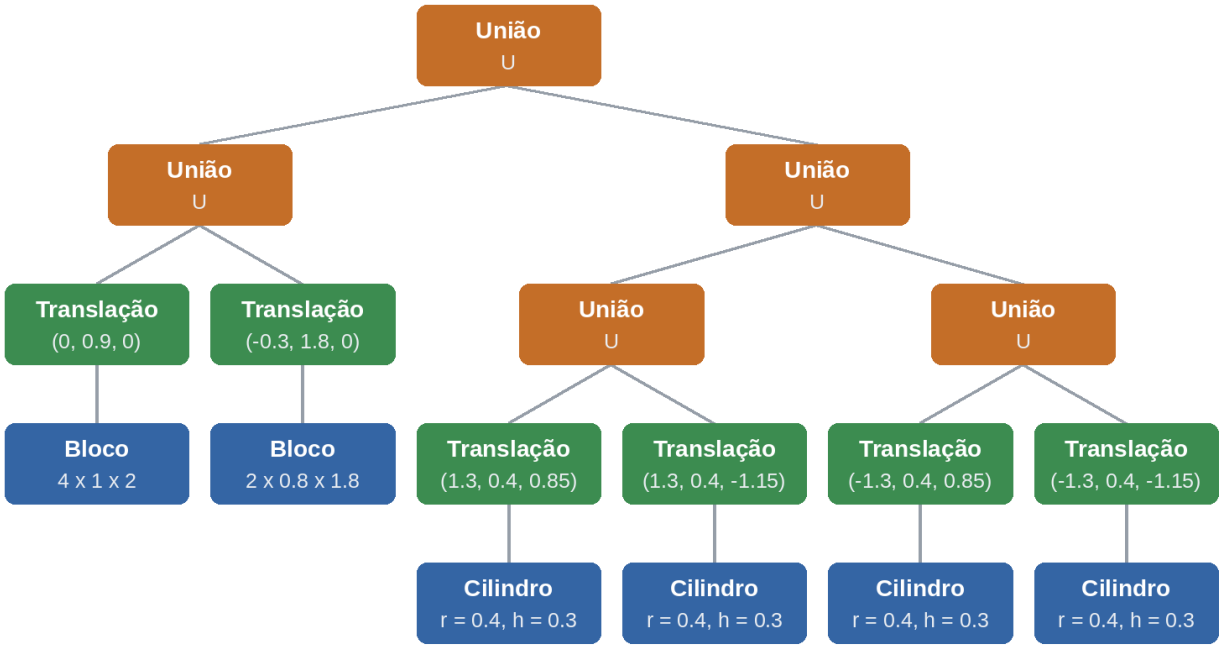
TArvoreCSG CriaOperacaoBooleana(
  EOperacaoBooleana operacao,
  TArvoreCSG esquerda,
  TArvoreCSG direita)
{
  return make_shared<const TNoCSG>(
    operacao, esquerda, direita);
}
```



- O nó não muda depois de criado: o ponteiro é para const e TNoCSG não tem cópia nem atribuição
- Uma operação só aloca o nó novo e aponta para as raízes das entradas; nada é copiado
- A mesma subárvore pode estar em várias árvores (desenho ao lado)
- O construtor rejeita subárvore vazia com invalid_argument: toda árvore criada é bem formada
- Na interface, cada operação cria um modelo novo; remover um modelo não afeta os que foram feitos a partir dele

Árvore CSG

Exibição do diagrama



Árvore do carro: uniões (laranja), translações (verde) e primitivas (azul)

MontaDiagrama, em pré-ordem

- Nível = profundidade do nó; define a linha do desenho
- Folhas recebem colunas consecutivas, na ordem de visita
- Movimento rígido fica na coluna do filho
- Operação booleana fica no meio das colunas dos dois filhos
- Cada caixa tem o tipo e os parâmetros do nó, com uma cor por tipo

Interface

- O painel converte nível e coluna em posição e desenha com o Dear ImGui
- A rolagem começa centrada na raiz

Primitivas

Contrato comum

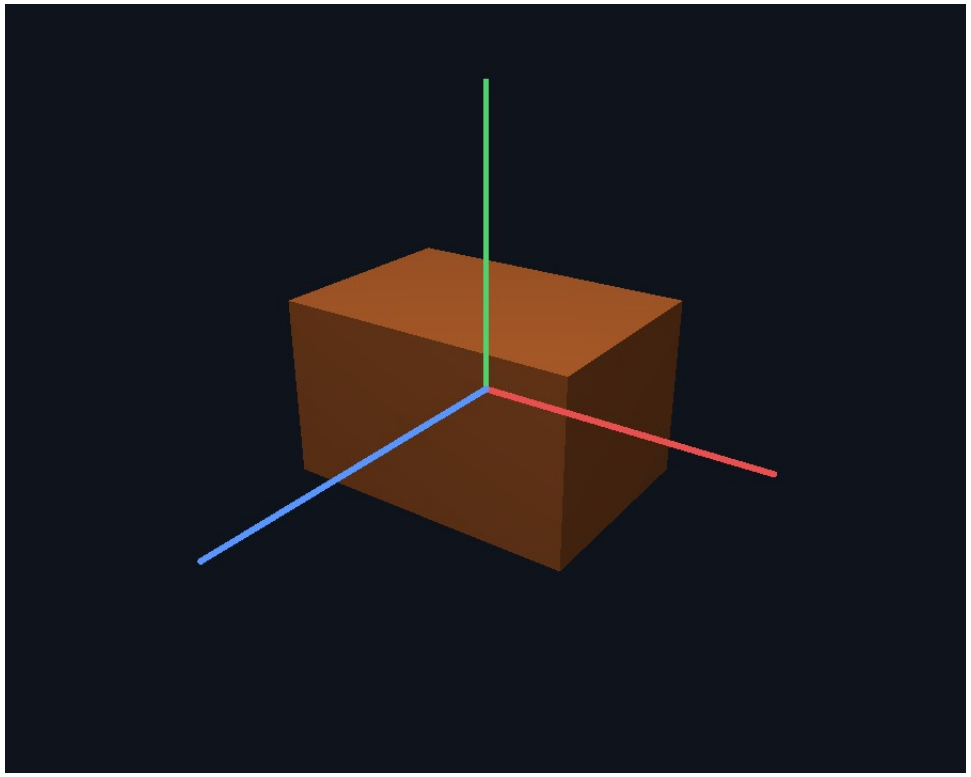
```
class TPrimitiva {
  Tipo()
  ClassificaPonto(ponto)
  ClassificaAresta(aresta)
  Normal(ponto)
  Escalada(fatores)
  CaixaCanonica()
  ArestasDoContorno()
};
```

Método	Usado por
ClassificaPonto	Octree; normal em operação booleana; aramado
ClassificaAresta	Classificação de arestas: ray-casting, aramado, interface
Normal	Iluminação do ray-casting
Escalada	Operação de escala
CaixaCanonica	Caixa envolvente: octree e câmera
ArestasDoContorno	Arestas candidatas do aramado

Distância com sinal: menor que -10^{-9} é IN, maior que 10^{-9} é OUT, senão ON (TOLERANCIA_GEOMETRICA)

Primitivas

Bloco



Bloco 2 × 1 × 1,5; eixos x, y e z em vermelho, verde e azul

$$|x| \le a/2 \qquad |y| \le b/2 \qquad |z| \le c/2$$

Ponto: excesso = max(|x| − a/2, |y| − b/2, |z| − c/2), usado como distância com sinal

Normal: eixo de maior excesso, com o sinal da coordenada; por exemplo (±1, 0, 0)

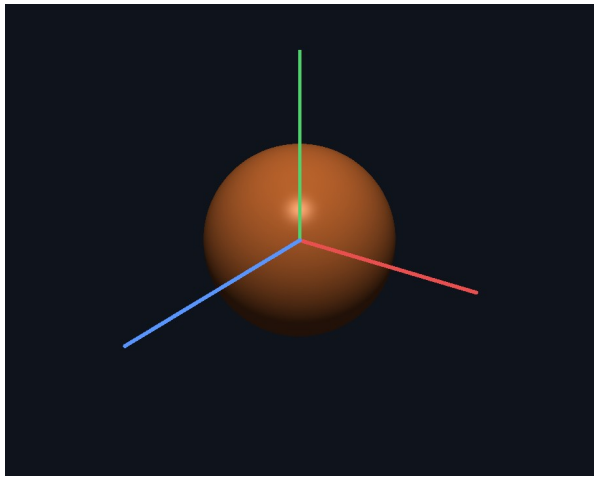
Caixa canônica: de (−a/2, −b/2, −c/2) a (a/2, b/2, c/2)

Arestas do contorno: os 12 lados ligam cantos cujos índices de 3 bits (um por eixo) diferem em um único bit

Validação: lados devem ser finitos e positivos

Primitivas

Esfera e cilindro



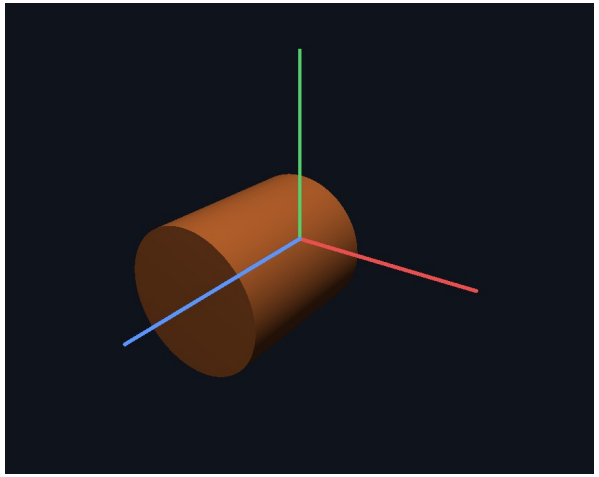
Esfera de raio r

Semiespaço: $x^2 + y^2 + z^2 - r^2 \leq 0$

Distância com sinal: $|p| - r$

Normal: $p \,/\, |p|$

Caixa: de (−r, −r, −r) a (r, r, r)



Cilindro de raio r e altura h

Semiespaços: $C = H1 \cap H2 \cap H3$

$H1: x^2 + y^2 - r^2 \leq 0 \quad H2: z \geq 0 \quad H3: z - h \leq 0$

Excesso: $\max(\sqrt{(x^2 + y^2)} - r, -z, z - h)$

Normal: (x, y, 0) / √(x² + y²) na lateral; ±z nas bases

Movimentos rígidos

Matrizes em coordenadas homogêneas

```
T(d) = | 1  0  0  dx |
       | 0  1  0  dy |
       | 0  0  1  dz |
       | 0  0  0  1  |

Rz(θ) = | cos θ  −sen θ  0  0 |
        | sen θ   cos θ  0  0 |
        |   0       0    1  0 |
        |   0       0    0  1 |

R = Rz(θz) · Ry(θy) · Rx(θx)
```

- Matrizes 4×4 (TMatriz4x4): translação e rotação viram a mesma operação
- Rotação com três ângulos em graus, aplicados em torno de x, depois y, depois z (Mortenson, apêndice C)
- Ângulo positivo no sentido anti-horário, pela regra da mão direita
- Inversas sem inverter matriz:

  $T(d)^{-1} = T(-d)$

  $R^{-1} = Rx(-\theta x) \cdot Ry(-\theta y) \cdot Rz(-\theta z)$
- Matriz() leva do sistema do filho ao do pai; MatrizInversa(), do pai ao filho
- Valores não finitos são rejeitados na criação do movimento

Movimentos rígidos

A aresta no sistema do filho

Classificar contra o filho exige a aresta no sistema dele. Em vez de mover o sólido, a aresta é movida pela inversa:

$$\mathbf{E'(t) = M^{-1} \cdot E(t)}$$

- M^{-1} é afim: basta transformar as duas extremidades
- O parâmetro se mantém:

$$M^{-1}(p + t\,d) = M^{-1}p + t \cdot M^{-1}d$$

- Os trechos devolvidos pelo filho valem sem conversão para a aresta original
- O mesmo para pontos: ClassificaPonto(filho, M^{-1} p)

```
TAresta LevaAoSistemaDoFilho(aresta) {
  inversa = MatrizInversa();
  return TAresta(
    inversa.TransformaPonto(aresta.Inicio()),
    inversa.TransformaPonto(aresta.Fim()));
}
```

```
case ETipoNoCSG::MOVIMENTO_RIGIDO:
  // trechos do filho valem para a aresta original
  return ClassificaAresta(
    arvore->Filho(),
    movimento.LevaAoSistemaDoFilho(aresta));
```

Caso do movimento rígido na classificação da aresta (simplificado)

Movimentos rígidos

Normais pela transposta da inversa

As normais (da regularização e do ray-casting) são calculadas no sistema do filho e precisam voltar ao do pai.

- Uma direção tangente t vai ao pai por M: $t' = M\,t$
- A normal precisa continuar perpendicular: $n'^{T}\,t' = 0$
- Com $n' = (M^{-1})^{T}\,n$:

$$n'^{T}\,t' = n^{T}\,M^{-1}\,M\,t = n^{T}\,t = 0$$

- Só a parte 3×3 entra: normal é direção, a translação não a afeta
- Depois de transformada, a normal é normalizada
- Para rotação pura, $(M^{-1})^{T}$ coincide com M

```
// elementos de MatrizInversa()
for (i = 0; i < 3; ++i)
  for (k = 0; k < 3; ++k)
    resultado[i] +=
      elementos[k][i] * normal[k];
```

TransformaNormalPelaTransposta: índices trocados, k e i

```
normal = Normaliza(
  movimento.MatrizInversa()
    .TransformaNormalPelaTransposta(n));
```

Escala

Escala pelos parâmetros da primitiva

A escala muda os parâmetros de dimensão: ela não é nó. Escala(árvore, fatores) desce a árvore:

- Primitiva: troca por Escalada(fatores), com os parâmetros multiplicados
- Movimento rígido: recria o mesmo movimento sobre o filho escalado
- Operação booleana: rejeitada com invalid_argument

```
Escala(T(5,0,0) · Bloco(2,2,2), (2,1,1))
  = T(5,0,0) · Bloco(4,2,2)
```

Os fatores valem nos eixos do sistema canônico da primitiva. A posição não muda: o bloco continua centrado em x = 5.

Primitiva	Novos parâmetros	Restrição
Bloco	a·sx, b·sy, c·sz	—
Esfera	r·s	sx = sy = sz
Cilindro	r·sx, h·sz	sx = sy

```
switch (arvore->Tipo()) {
  case PRIMITIVA:
    return CriaPrimitiva(
      primitiva.Escalada(fatores));
  case MOVIMENTO_RIGIDO:
    return CriaMovimentoRigido(movimento,
      Escala(arvore->Filho(), fatores));
```

Classificação SMC

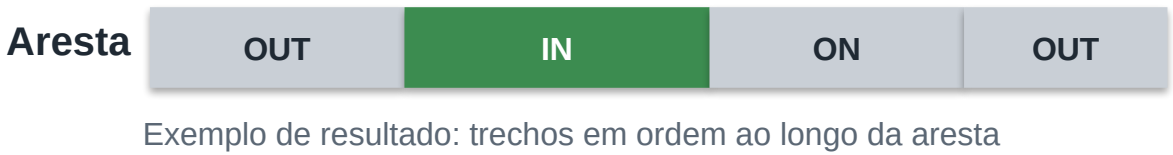
O que é e como está dividida

Classificar uma aresta contra um sólido é dividi-la em trechos, cada um com uma classe:

IN: no interior do sólido

ON: sobre a fronteira

OUT: fora do sólido



Exemplo de resultado: trechos em ordem ao longo da aresta

Usada pelo ray-casting (o raio é uma aresta), pelo aramado e pela exibição de arestas classificadas na interface.

Prim_M

Classifica a aresta contra uma primitiva, resolvendo as equações dela

Combine_M

Junta as classificações contra os dois filhos de uma operação booleana, trecho a trecho

Tree_M

Percorre a árvore: Prim_M nas folhas, desce pelos movimentos rígidos e chama Combine_M nas operações

Tree_M

Classificação da aresta por divisão e conquista

```
Tree_M(S, E):
  se S é primitiva:
    retorna Prim_M(S, E)
  se S é movimento rígido M:
    retorna Tree_M(S.filho, M⁻¹ · E),
            normais levadas por (M⁻¹)ᵀ
  se S é operação booleana:
    retorna Combine_M(Tree_M(S.esq, E),
                      Tree_M(S.dir, E), S.op)
```

- Cada nó é visitado uma vez por aresta
- A mesma função atende a exibição da SMC, o ray-casting e o aramado

Aresta: $E(t) = p + t\,(q - p), \quad t \in [0, 1]$

```
struct TSegmentoClassificado {
  double tInicio, tFim;
  EClassificacao classificacao;
  TVizinhanca vizinhanca;  // regularização
};
```

Resultado (TClassificacaoAresta)

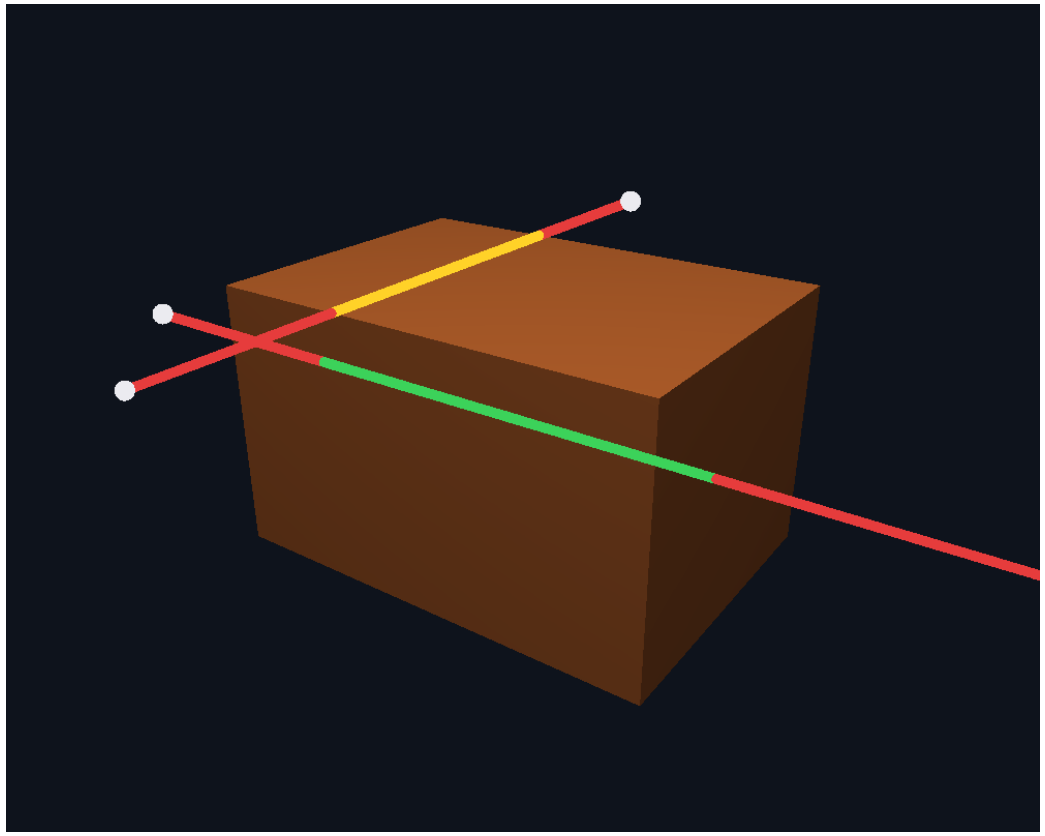
- Trechos em ordem de t, sem sobreposição, cobrindo [0, 1]
- Cada trecho é IN, ON ou OUT
- Trechos vizinhos iguais são fundidos em um só

Prim_M

Aresta contra o bloco

$t_1 = (-a/2 - p_x) / d_x \qquad t_2 = (a/2 - p_x) / d_x$
intervalo em x: $[\min(t_1, t_2), \max(t_1, t_2)]$

- Interseção dos três intervalos: entrada = maior início, saída = menor fim
- Recorte em [0, 1]: OUT antes, IN ou ON no intervalo, OUT depois
- $d_x = 0$: aresta paralela à faixa; inteira dentro, inteira fora ou sobre um dos planos
- Sobre um plano o trecho é ON e guarda a normal dele; sobre um lado do bloco, duas normais
- Intervalo de comprimento $\leq 10^{-9}$ (encostar num canto) fica OUT



Prim_M no bloco: OUT em vermelho, IN em verde, ON em amarelo

Prim_M

Aresta contra a esfera e o cilindro

Esfera

$$(d \cdot d)\, t^2 + 2\,(p \cdot d)\, t + (p \cdot p - r^2) = 0$$

- Discriminante ≤ 0: a reta não corta ou só tangencia; tudo OUT
- Duas raízes $t_1 < t_2$: IN entre elas, recortado em [0, 1]
- Uma aresta reta nunca fica sobre a esfera: não há trecho ON

Cilindro

$$\begin{aligned} &\text{faixa:} && 0 \le p_z + t\, d_z \le h \\ &\text{lateral:} && (d_x^2 + d_y^2)\, t^2 + 2\,(p_x d_x + p_y d_y)\, t \\ &&& + (p_x^2 + p_y^2 - r^2) = 0 \end{aligned}$$

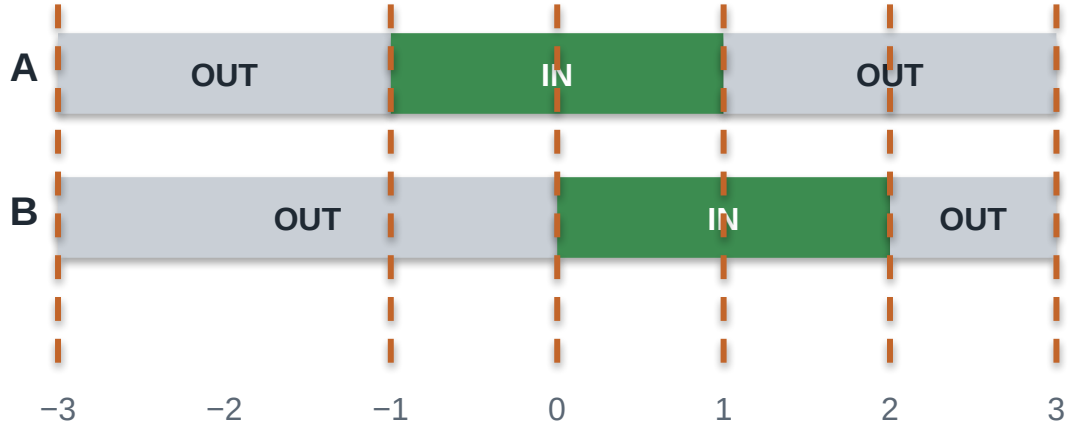
- Intervalo final: interseção do intervalo da faixa com o da lateral
- Paralela ao eixo (dx = dy = 0): dentro, fora ou sobre a lateral pela distância ao eixo; sobre ela, ON com normal radial
- Paralela às bases sobre z = 0 ou z = h: ON com normal −z ou +z

Combine_M

Passo 1: pontos de corte

Exemplo dos quatro passos: blocos de lado 2, A centrado na origem e B deslocado de 1 em x; aresta de x = −3 a x = 3.

- Junta inícios e fins dos trechos dos dois lados, mais 0 e 1
- Recorta em [0, 1] e ordena
- Remove repetições: um corte a menos de 10^{-9} do anterior é o mesmo



x ao longo da aresta; linhas tracejadas: pontos de corte

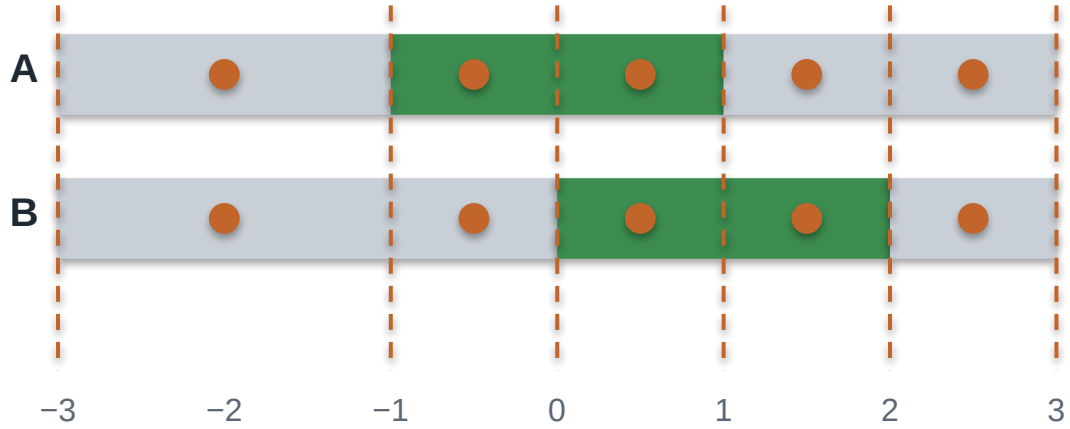
```
Cortes do exemplo:
  t = 0, 1/3, 1/2, 2/3, 5/6, 1
  x = −3, −1,  0,  1,   2,  3
```

Combine_M

Passo 2: classe de cada trecho nos dois lados

- Entre cortes consecutivos nenhuma das classificações muda: todo ponto de mudança já é corte
- Basta um ponto interno; usamos o ponto médio
- SegmentoEm(classificação, t) devolve o trecho do filho que contém t

Trecho de t	Meio	A	B
[0, 1/3]	1/6	OUT	OUT
[1/3, 1/2]	5/12	IN	OUT
[1/2, 2/3]	7/12	IN	IN
[2/3, 5/6]	3/4	OUT	IN
[5/6, 1]	11/12	OUT	OUT



Pontos: o meio de cada trecho; verde é IN, cinza é OUT

Combine_M

Passo 3: tabela da operação

União

∪	IN	ON	OUT
IN	IN	IN	IN
ON	IN	ON*	ON
OUT	IN	ON	OUT

Interseção

∩	IN	ON	OUT
IN	IN	ON	OUT
ON	ON	ON*	OUT
OUT	OUT	OUT	OUT

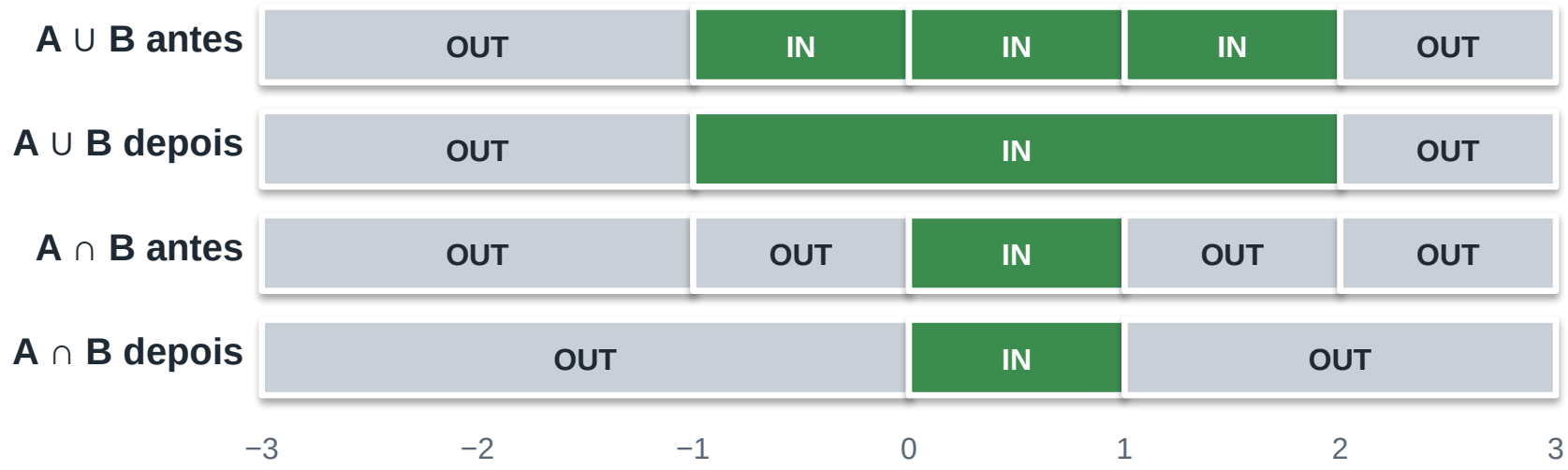
- União: IN se algum lado é IN; senão ON se algum é ON; senão OUT
- Interseção: OUT se algum lado é OUT; IN se os dois são IN; senão ON
- ON de um só lado: o trecho leva a vizinhança desse lado
- * ON com ON é decidido pela regularização

Trecho de t	A	B	A ∪ B	A ∩ B
[0, 1/3]	OUT	OUT	OUT	OUT
[1/3, 1/2]	IN	OUT	IN	OUT
[1/2, 2/3]	IN	IN	IN	IN
[2/3, 5/6]	OUT	IN	IN	OUT
[5/6, 1]	OUT	OUT	OUT	OUT

Tabela aplicada aos cinco trechos do exemplo

Combine_M

Passo 4: fusão de trechos vizinhos



Resultado:

A ∪ B: IN em x ∈ [−1, 2]

A ∩ B: IN em x ∈ [0, 1]

```
if (!resultado.empty() &&
    resultado.back().classificacao == trecho.classificacao &&
    MesmaVizinhanca(resultado.back().vizinhanca, trecho.vizinhanca))
  resultado.back().tFim = trecho.tFim;
else
  resultado.push_back(trecho);
```

- Mesma classe e mesma vizinhança: um trecho só
- A lista fica mínima e não cresce a cada nível
- ON com vizinhanças diferentes não se fundem

Regularização

O problema de ON com ON

Operações regularizadas: o resultado é sempre um sólido, sem faces soltas ou internas. A tabela de pontos não garante isso:

- União de A e B encostados: a face comum está no interior da união, mas a tabela dá ON ∪ ON = ON
- Interseção de A e B encostados: os sólidos só se tocam e o resultado deveria ser vazio, mas a tabela dá ON
- Nos dois casos falta saber de que lado de cada face está o material

Escolhemos a regularização pela vizinhança: cada trecho ON carrega as direções que entram no sólido.

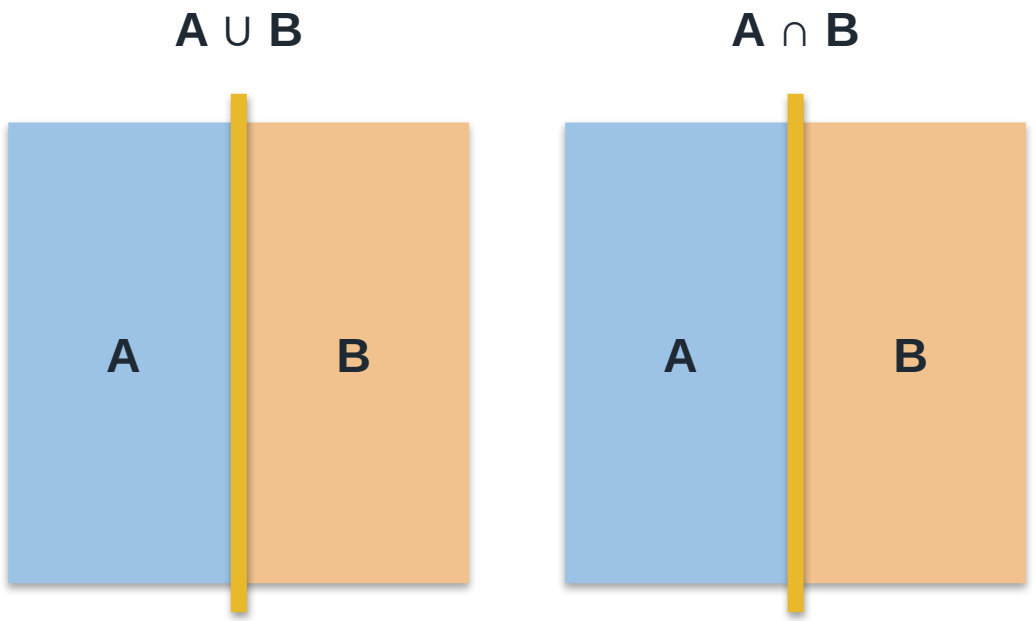


tabela: ON

esperado: IN

tabela: ON

esperado: OUT

Em amarelo, a aresta sobre a face comum, vista de lado

Regularização

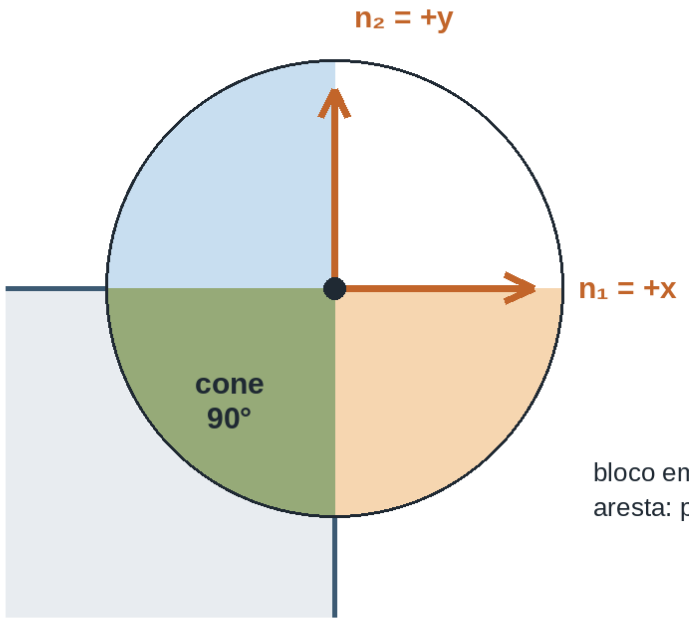
Vizinhança como cones de normais

Vizinhança: as direções que, a partir do ponto, entram no sólido.

- Cada face que contém o trecho tem normal exterior n; a direção v entra se $n \cdot v \leq 0$
- Cone (TConeVizinhanca): interseção desses semiespaços, um por normal
- Vizinhança (TVizinhanca): lista de cones, pois uma união pode ter material em vários

Quem preenche

- Bloco: uma normal sobre uma face; duas sobre um lado
- Cilindro: ±z nas bases; radial na lateral
- Movimento rígido: normais voltam ao pai por $(M^{-1})^T$
- Combine_M: decide ON com ON e passa adiante a vizinhança resultante



bloco em corte
aresta: ponto central

$n_1 \cdot v \leq 0$: semiplano à esquerda (azul) $n_2 \cdot v \leq 0$: semiplano abaixo (laranja)

Lado de um bloco: duas normais, cone de 90°

Regularização

Projeção no plano perpendicular à aresta

As normais das faces que contêm a aresta são perpendiculares a ela: no plano perpendicular, cada direção é um ângulo.

- Base (u, w): u = eixo × a, w = eixo × u, com a o eixo coordenado menos alinhado com a aresta
- Cada normal vira um ângulo: $\varphi = \text{atan2}(n \cdot w, n \cdot u)$
- $n \cdot v \le 0$ vira o semiplano oposto a n: arco de início $\varphi + \pi/2$ e abertura π
- O cone vira a interseção dos arcos das suas normais; arcos de até π se intersectam num único arco
- Cone com abertura $\le 10^{-9}$ é descartado

```
for (i = 0; i < cone.size(); ++i) {
  phi = atan2(cone[i]·w, cone[i]·u);
  semiplano = { phi + π/2, π };
  arco = i == 0
    ? semiplano
    : IntersectaArcos(arco, semiplano);
}
```

ArcosDaVizinhanca: um arco por cone

```
struct TArco { double inicio; double abertura; };
```

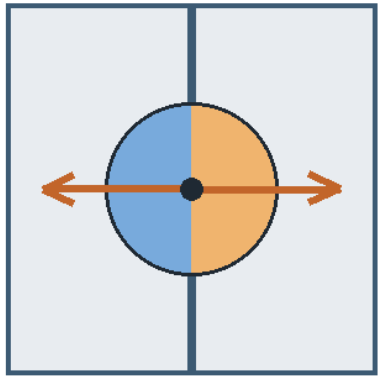
Regularização

União: os arcos cobrem 360°?

Interior da união: toda direção entra em A ou em B.

- Vizinhança unida: cones dos dois lados, sem repetir cones iguais (faces coplanares do mesmo lado)
- Arcos ordenados pelo início e percorridos guardando o alcance; um arco que começa depois do alcance é buraco
- A volta fecha: o trecho passa a IN
- Não fecha: continua ON, com a vizinhança unida, para os nós de cima

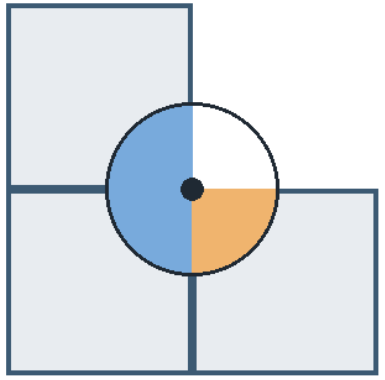
União, faces encostadas



cobre 360°: IN

Faces encostadas: dois semicírculos

União em L, canto côncavo



cobre 270°: ON

Canto côncavo de um L

```
if (CobreTodasAsDirecoes(ArcosDaVizinhanca(
      vizinhancaUnida, direcao)))
  return { ..., EClassificacao::IN };
```

Regularização

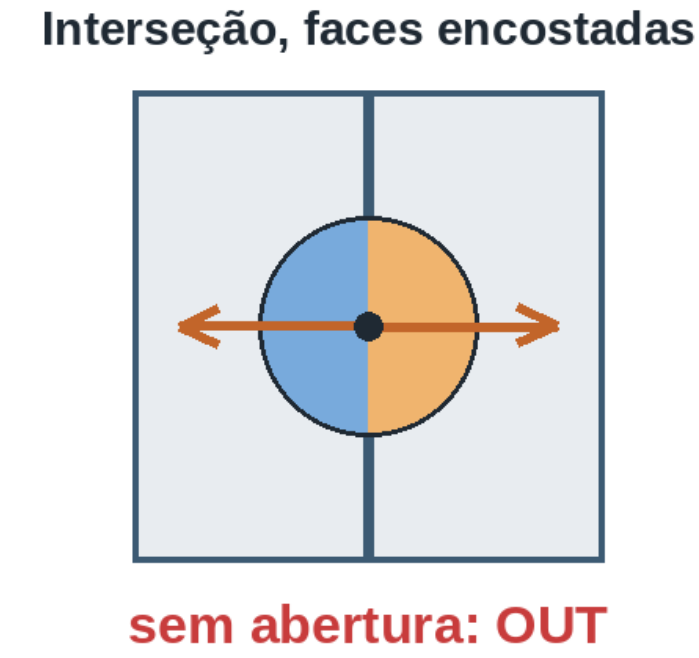
Interseção: cones intersectados

Interior da interseção: a direção entra em A e em B.

- Para cada par (cone de A, cone de B), o cone interseção tem as normais dos dois, sem repetir
- Pares sem abertura são descartados
- Nenhum par restante: o trecho passa a OUT
- Senão, continua ON com os cones que restaram
- Sem vizinhança conhecida de um dos lados: fica ON

Resultado nos dois casos do problema

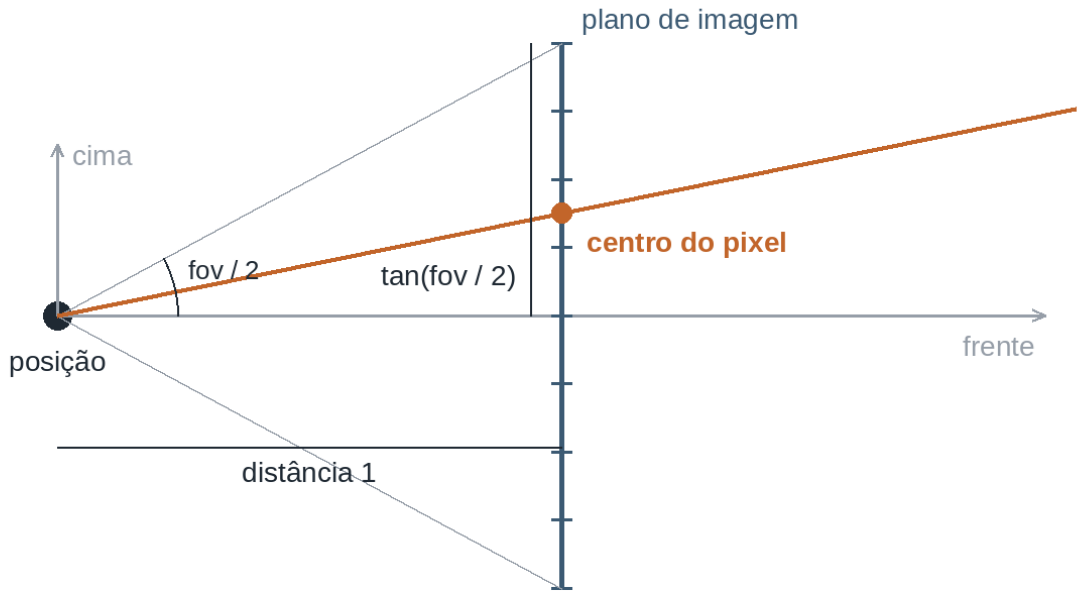
- A ∪ B encostados: face comum IN, aparece verde na exibição da SMC
- A ∩ B encostados: face comum OUT, a interseção é vazia



Semicírculos opostos: interseção sem abertura

Ray-casting

Câmera e raio de cada pixel



Corte vertical da câmera; o raio passa pelo centro de um pixel

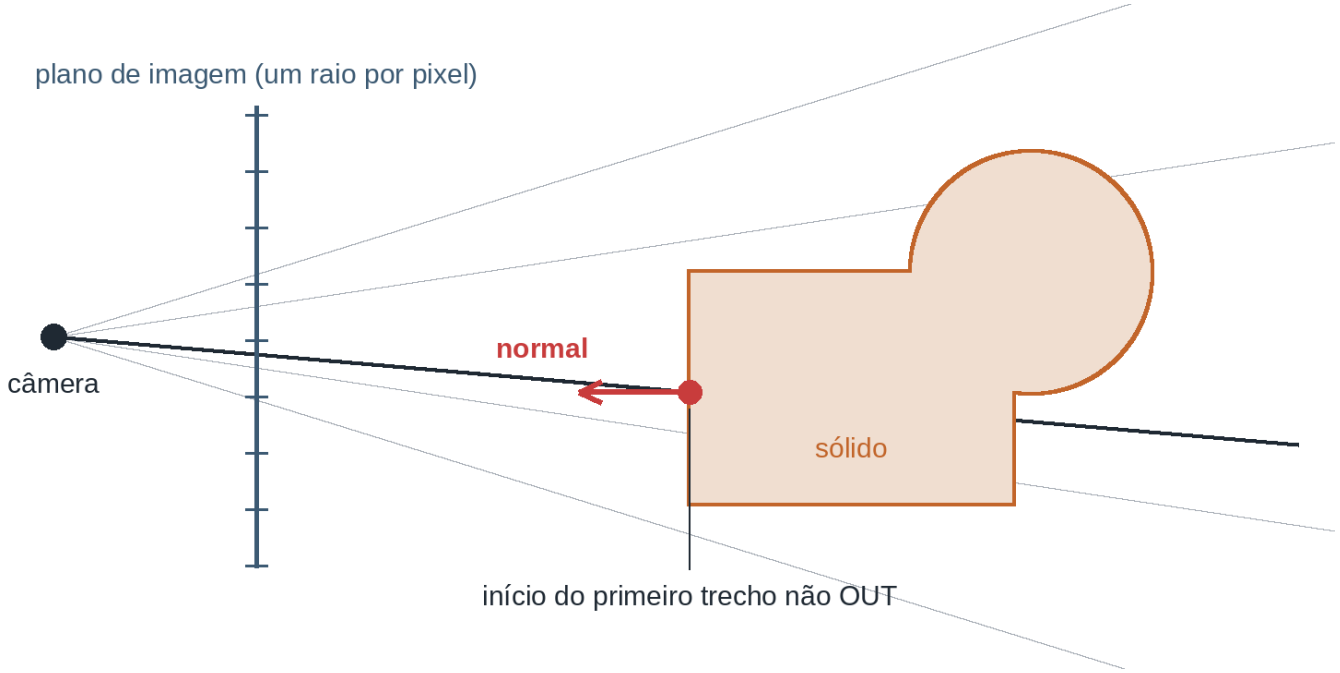
- frente = normaliza(alvo − posição)
- direita = normaliza(frente × cima)
- cima' = direita × frente
- meiaAltura = tan(fov / 2); meiaLargura = meiaAltura · L / A

```
x = (2 (c + 0,5) / L − 1) · meiaLargura
y = (1 − 2 (l + 0,5) / A) · meiaAltura
dir = normaliza(frente + x·direita + y·cima')
```

- O raio é a aresta de posição até posição + alcance · dir
- Câmera em órbita do centro do modelo: azimute, elevação e distância

Ray-casting

Primeira interseção



- Em vez da aresta semi-infinita, uma finita que passa de toda a cena:

  `alcance = distância + 2 · raioDaCena + 1`

- Tree_M classifica o raio contra a árvore inteira
- Ponto atingido: início do primeiro trecho não OUT
- Trecho que começa em t = 0: câmera dentro do sólido, sem interseção
- Só trechos OUT: cor de fundo
- Resultado: t, ponto e normal (TIntersecao)

```
for (segmento : ClassificaAresta(arvore, raio))
  if (segmento.classificacao != OUT)
    return segmento.tInicio > tol ? interseção : nada;
```

Ray-casting

Normal no ponto atingido

```
case OPERACAO_BOOLEANA:
  naEsquerda = ClassificaPonto(esq, ponto);
  naDireita  = ClassificaPonto(dir, ponto);
  UNIAO:
    sobreEsquerda = naEsquerda == ON
                 && naDireita != IN;
  INTERSECAO:
    sobreEsquerda = naEsquerda == ON
                 && naDireita != OUT;
  return NormalNoPonto(
    sobreEsquerda ? esq : dir, ponto);
```

NormalNoPonto desce até a primitiva em cuja fronteira o ponto está:

- Primitiva: a sua Normal(ponto)
- Movimento: normal do filho em M^{-1} p, levada por $(M^{-1})^T$ e normalizada
- União: fronteira de um filho e fora do interior do outro
- Interseção: fronteira de um filho e não fora do outro
- Se a esquerda não satisfaz, vem da direita

Face plana: normal constante. Esfera e lateral do cilindro: normal variável, sombreado suave.

Iluminação de Phong

Modelo e parâmetros

$$I = k_a \cdot C + k_d \cdot \max(0, n \cdot l) \cdot (C \odot L) + k_s \cdot \max(0, r \cdot v)^{\alpha} \cdot L, \qquad r = 2\,(n \cdot l)\,n - l$$

Vetores

- n: normal unitária no ponto
- l: direção do ponto para a luz
- v: direção do ponto para o observador
- r: reflexão de l em torno de n
- $C \odot L$: produto componente a componente

Parâmetros e detalhes

- Material: C = (0,85; 0,45; 0,2), ka = 0,15, kd = 0,7, ks = 0,3, $\alpha = 32$
- Luz branca, acima da câmera, que a acompanha na órbita
- Especular só com $n \cdot l > 0$: sem brilho em face contra a luz
- Cada componente limitado a [0, 1] e convertido em byte

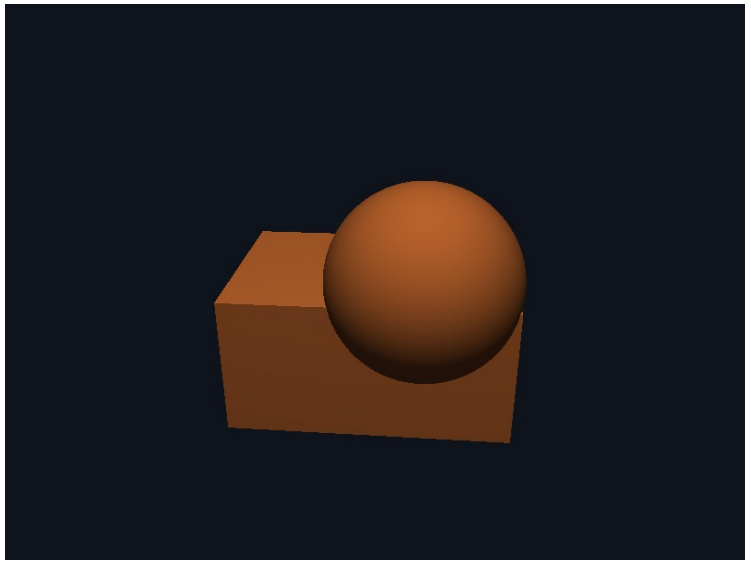
Iluminação de Phong

Contribuição de cada termo



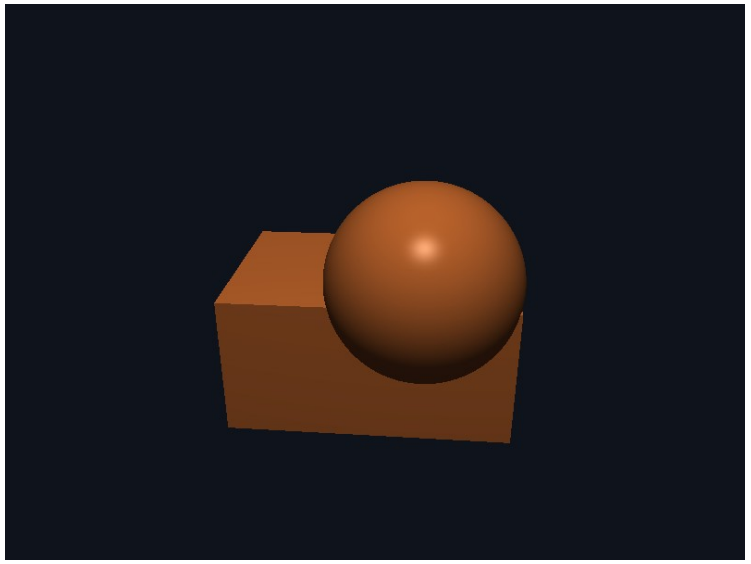
Ambiente

ka·C: cor constante, sem noção de forma



Ambiente + difusa

kd·max(0, n·l): depende do ângulo com a luz; mostra a forma



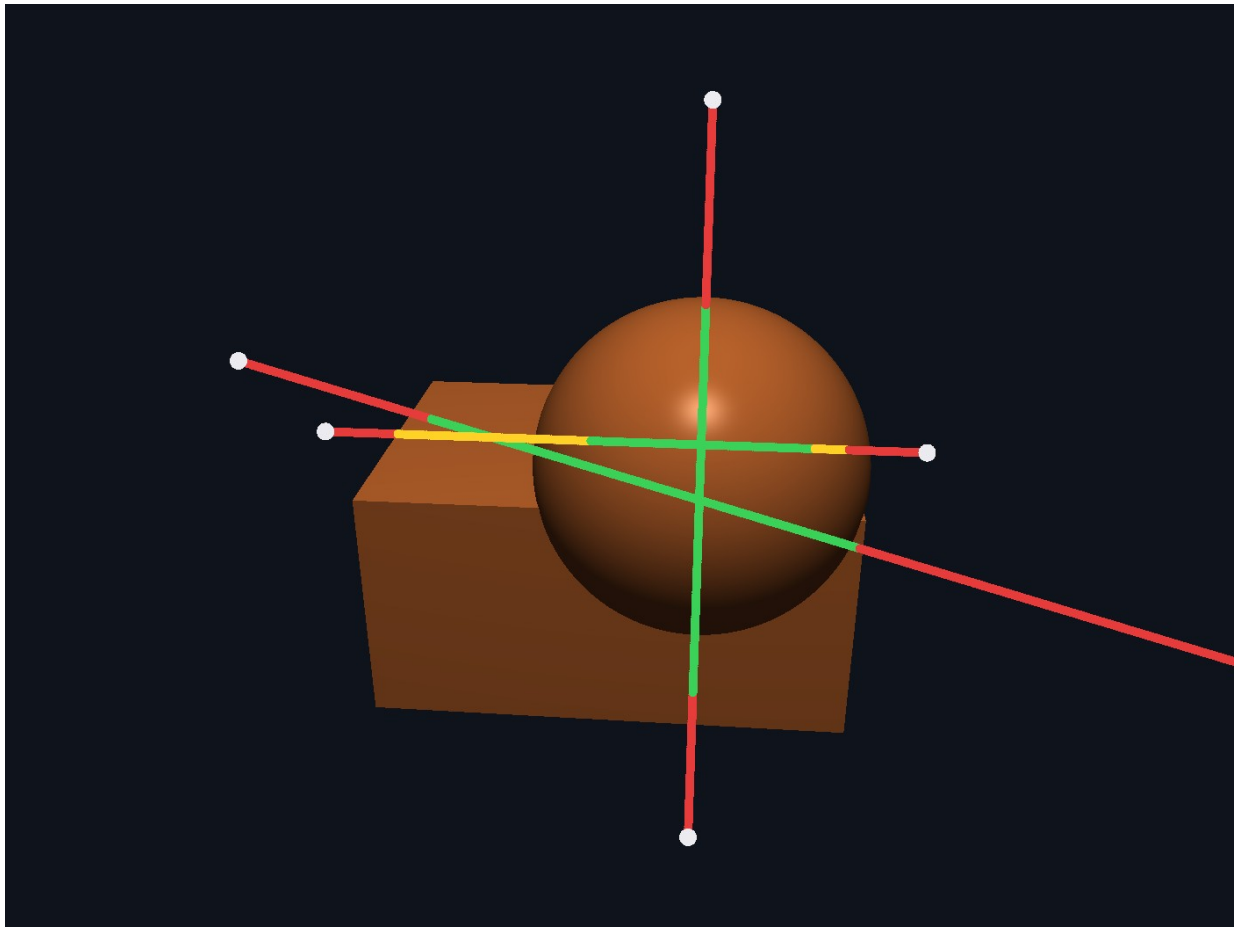
Ambiente + difusa + especular

ks·max(0, r·v)^α: depende do observador; brilho

Um raio pelo centro de cada pixel; a imagem só é recalculada quando a árvore ou a câmera mudam.

Exibição da SMC

Arestas de teste na interface



- Três arestas iniciais, paralelas aos eixos, atravessando a caixa do modelo com folga de um quarto de cada lado
- Início e fim editáveis; arestas podem ser acrescentadas e removidas
- Cada aresta é classificada por Tree_M contra a árvore inteira, só quando a árvore ou as arestas mudam
- O painel lista os trechos: t inicial, t final e classe

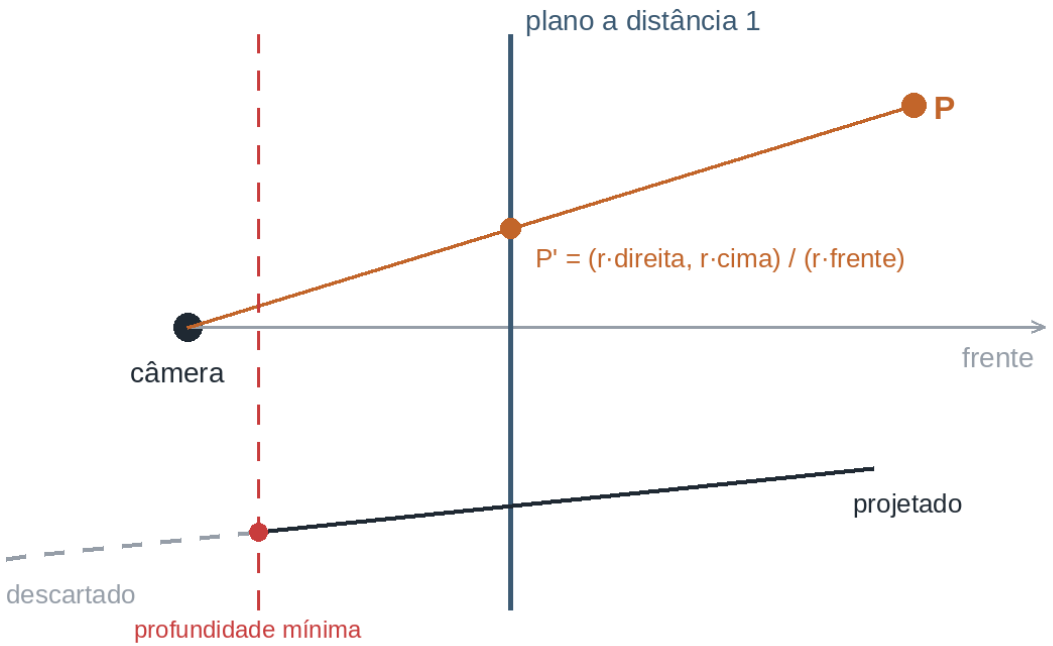
IN

ON

OUT

Exibição da SMC

Projeção e recorte



Projeção: inverso do raio do pixel

```
r = P − posição;  prof = r · frente
x = (r · direita) / prof
y = (r · cima') / prof
coluna = (x / meiaLargura + 1) · L / 2
```

Recorte antes de projetar

- prof ≤ 0 inverte ou explode a divisão: recorta em prof = 10^{-3}
- prof varia linearmente com t:

  $t = (\text{mín} - \text{prof}_0) / (\text{prof}_1 - \text{prof}_0)$
- Linhas por cima da imagem, sem ocultação, para mostrar os trechos IN

Aramado

O que são os segmentos EonS

Classificar uma aresta E contra um sólido S a divide em três conjuntos:

`EinS` (E in S): parte no interior de S

`EonS` (E on S): parte sobre a fronteira de S

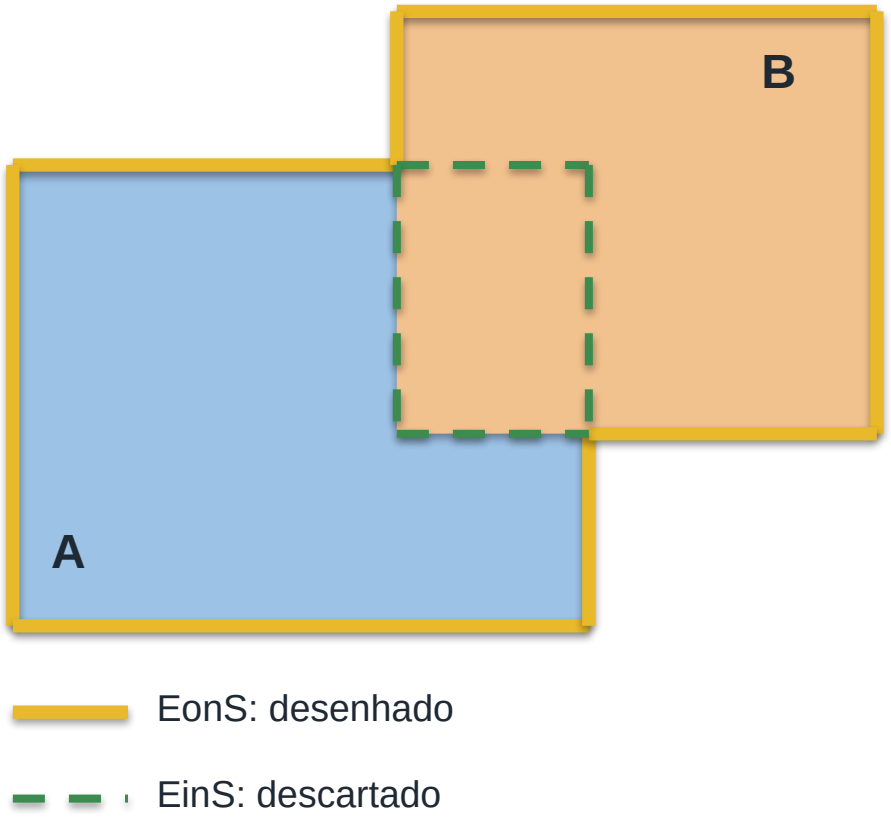
`EoutS` (E out S): parte fora de S

São os trechos IN, ON e OUT da classificação.

Aramado

- As arestas de um sólido estão na fronteira dele
- Candidatas: arestas das primitivas, que podem ser do sólido final
- Classifica cada candidata contra a árvore inteira e desenha só a parte EonS

A ∪ B em corte; lados classificados contra a união



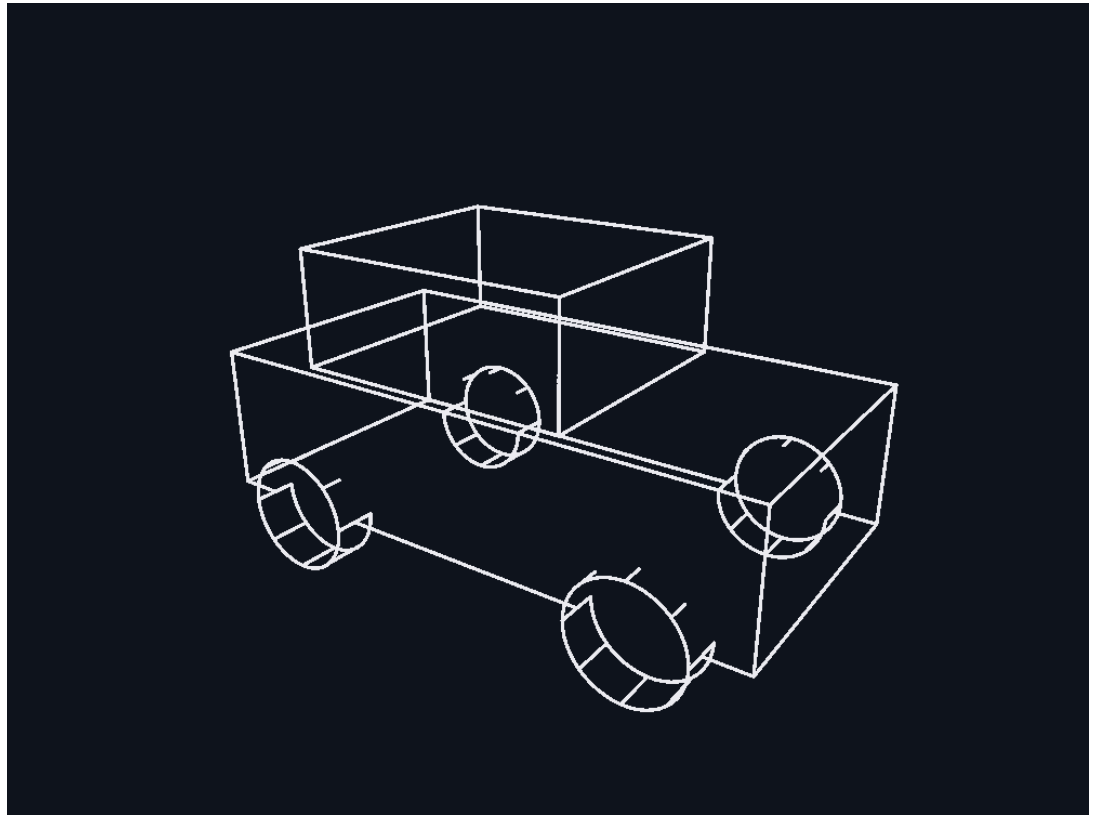
Aramado

Arestas candidatas

As candidatas vêm das primitivas. ArestasCandidatas(árvore) desce a árvore:

- Primitiva: ArestasDoContorno(), em posição canônica
- Movimento rígido: as do filho, levadas ao pai pela matriz M (a entidade sobe na árvore, ao contrário da SMC)
- Operação booleana: as dos dois filhos, concatenadas

Primitiva	Candidatas
Bloco	12 lados
Cilindro	2 círculos de 48 cordas e 8 geratrizes
Esfera	6 meridianos e 5 paralelos (−60° a 60°), 48 cordas cada



Aramado do carro do tema

Aramado

Classificação das candidatas

Aresta reta

- Tree_M contra a árvore inteira; ficam os trechos ON
- Parte dentro de outra primitiva sai IN; parte fora do resultado sai OUT; as duas somem

Curva aproximada por cordas

- Cordas de 1/48 de volta passam por dentro da primitiva: Tree_M daria IN, nunca ON
- Cada corda guarda o ponto da curva no meio dela (pontoDaCurva), que está na fronteira
- A corda é mantida se esse ponto for ON na árvore

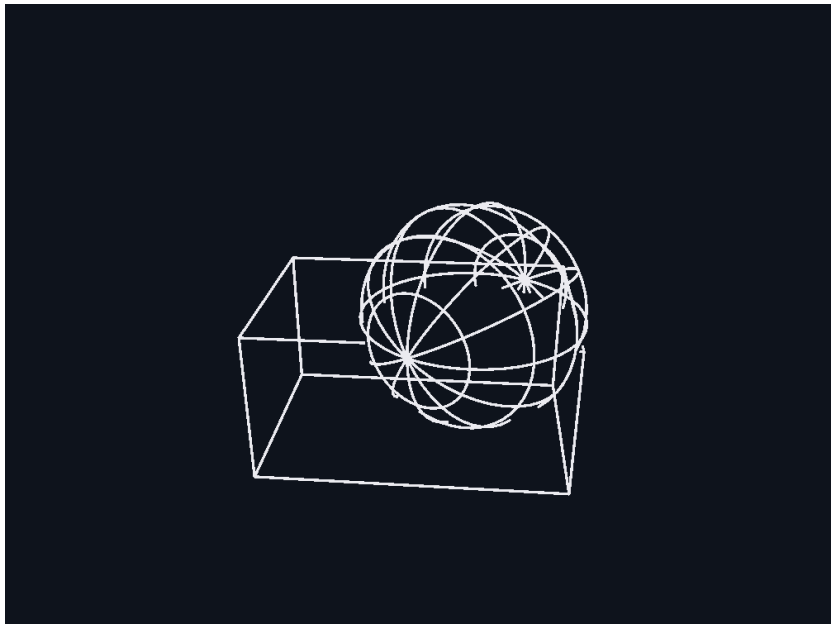
Segmentos calculados quando a árvore muda; projetados a cada quadro, como na SMC.

```
for (candidata : ArestasCandidatas(arvore)) {
  if (candidata.pontoDaCurva) {
    if (ClassificaPonto(arvore,
          *candidata.pontoDaCurva) == ON)
      segmentos.push_back(candidata.aresta);
    continue;
  }
  for (trecho : ClassificaAresta(arvore,
                  candidata.aresta))
    if (trecho.classificacao == ON)
      segmentos.push_back(parte do trecho);
}
```

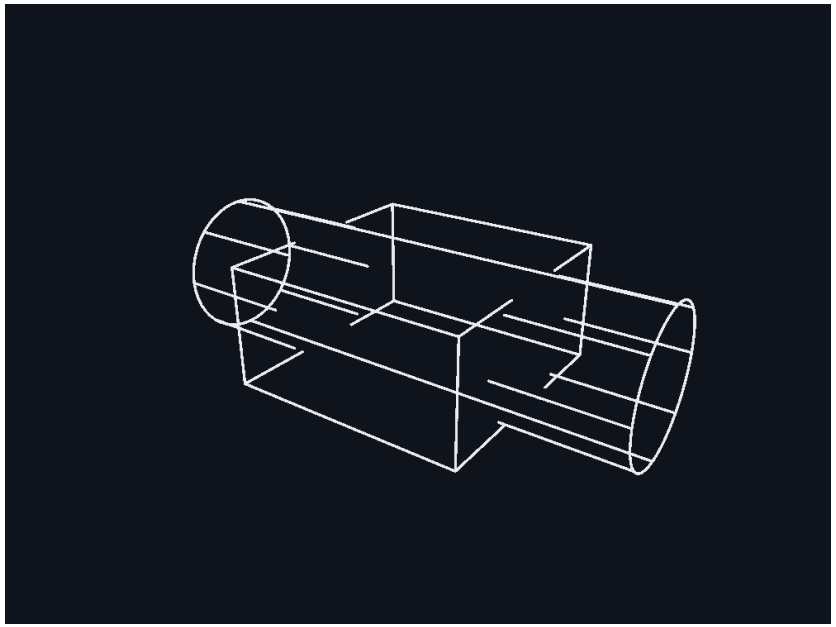
SegmentosEonS

Aramado

Resultados e limitações



Bloco girado unido a uma esfera



Cilindro atravessando um bloco

Limitações

- Curvas de interseção entre sólidos não aparecem
- Sem remoção de linhas ocultas
- Meridianos e paralelos não são arestas do sólido
- A corda pode passar um pouco do ponto em que a curva sai do sólido

No cilindro atravessando o bloco, faltam as duas circunferências onde a lateral fura as faces do bloco.

Volume

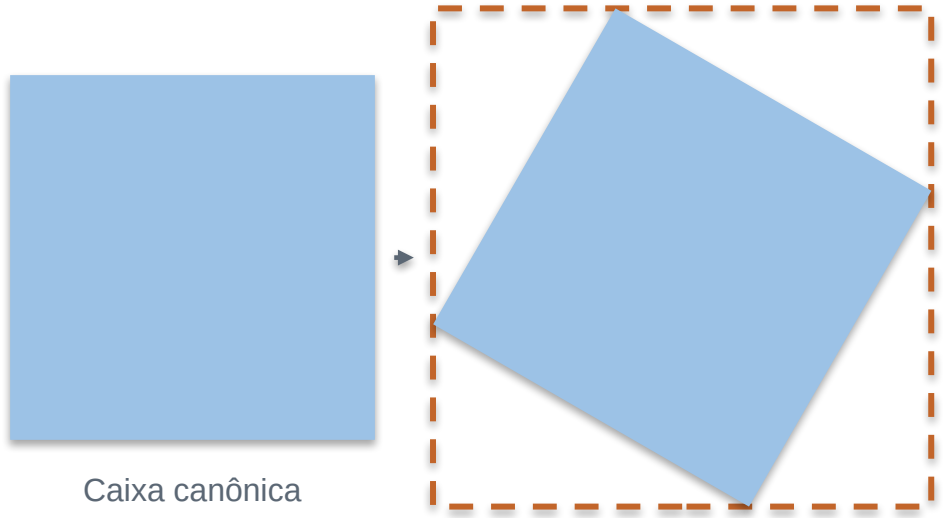
Caixa envolvente

A octree precisa de uma caixa inicial que contenha o sólido.
CaixaEnvolvente(árvore):

- Primitiva: caixa canônica
- Movimento rígido: transforma os 8 cantos da caixa do filho por M; caixa que os contém
- União: menor caixa que contém as duas
- Interseção: interseção das duas caixas

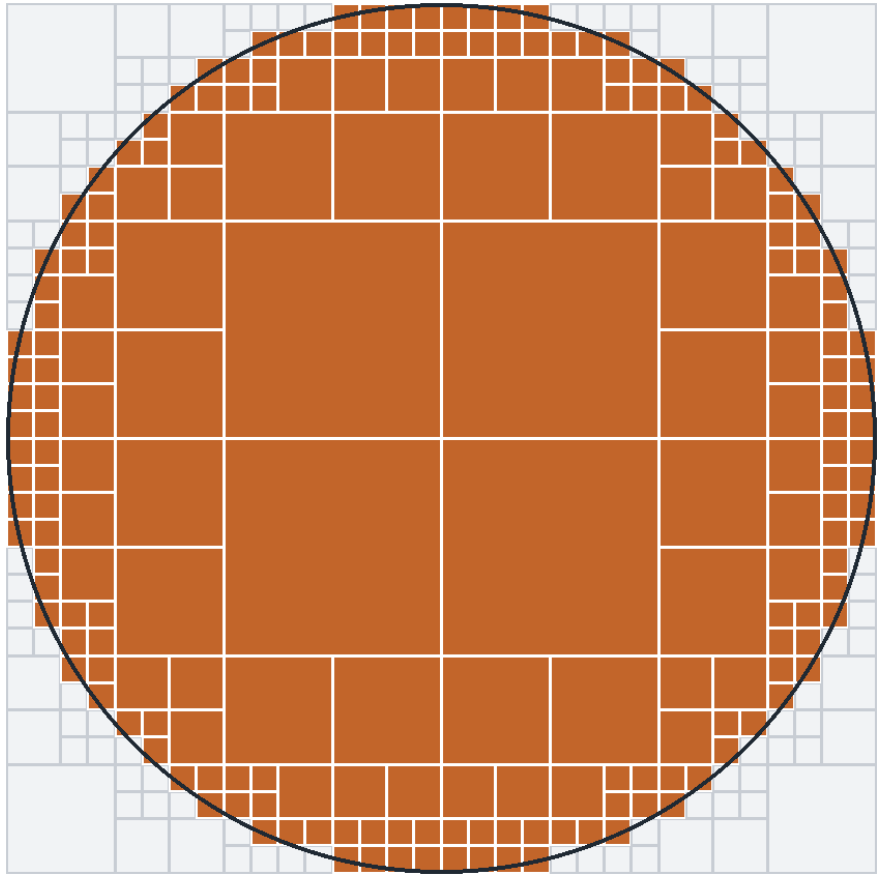
A caixa pode ser maior que o necessário (uma caixa girada cresce), mas sempre contém o sólido.

A mesma caixa centra a câmera e define as arestas iniciais da SMC.



Volume

Conversão da árvore em octree



Corte em z = 0 da octree da esfera, profundidade 5

- Cada célula classifica 9 pontos: o centro e os 8 cantos
- Todos IN ou ON: cheia. Todos OUT: vazia
- Misto: divide em 8 pelo centro; os 3 bits do índice escolhem a metade em x, y e z
- Na profundidade máxima, decide pelo centro: IN ou ON é cheia
- Volume = soma dos volumes das células cheias

```
subMin = { (i & 1) ? meio.x : min.x,
           (i & 2) ? meio.y : min.y,
           (i & 4) ? meio.z : min.z };
subMax = { (i & 1) ? max.x : meio.x, ... };
```

Volume

Precisão e custo

Profundidade	Volume	Erro	Tempo
2	4,0000	−4,51%	< 0,1 ms
4	4,2500	1,46%	0,9 ms
5	4,2129	0,58%	5 ms
6	4,1924	0,09%	18 ms
8	4,1885	−0,01%	290 ms

Esfera de raio 1; volume exato 4π/3 ≈ 4,1888

- O erro vem das células da fronteira, decididas pelo centro na profundidade máxima
- Abaixo de 1% a partir da profundidade 5
- Células na fronteira crescem cerca de 4× por nível (acompanham a área); o tempo também
- Na interface, profundidade de 1 a 8; fora disso, invalid_argument
- Nove pontos podem perder um detalhe menor que a célula; a profundidade limita esse erro

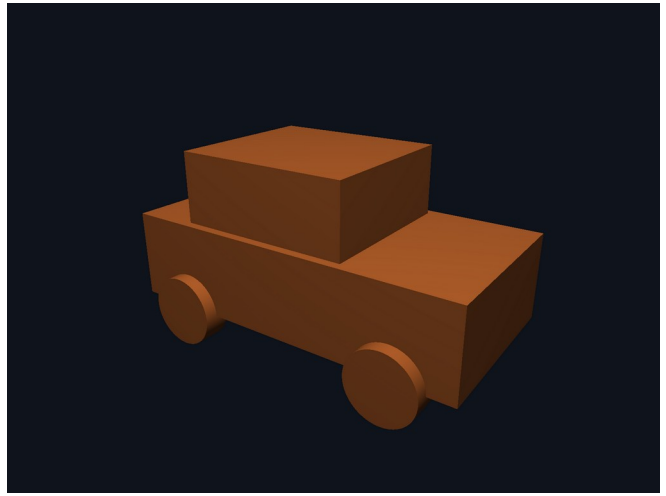
Persistência

Formato de arquivo e o modelo do tema

```
No1; B; S 4,1,2; T 0,0.9,0
No2; B; S 2,0.8,1.8; T -0.3,1.8,0
No3; C; S 0.4,0.3; T 1.3,0.4,0.85
No4; C; S 0.4,0.3; T 1.3,0.4,-1.15
No5; C; S 0.4,0.3; T -1.3,0.4,0.85
No6; C; S 0.4,0.3; T -1.3,0.4,-1.15
No7; U No1 No2
No8; U No3 No4
No9; U No5 No6
No10; U No8 No9
No11; U No7 No10
```

modelos/ford.txt

- Uma linha por nó; campos separados por ponto e vírgula; raiz na última linha
- Primitiva: B, E ou C, seguida de S com os parâmetros (lados; raio; raio e altura)
- Operação: U, I ou D seguida dos nomes de dois nós já definidos
- T e R: movimentos aplicados na ordem em que aparecem



Persistência

Leitura e escrita

Leitura

- Cada linha vira um nó; T e R viram nós de movimento acima dele, na ordem em que aparecem
- Só referencia nós já definidos: não há ciclos
- Ponto decimal independente da configuração regional
- Rejeita código desconhecido, nó não definido, nome repetido e número errado de valores, com a linha do erro

Escrita (CodificaNo)

- Escreve antes os nós referenciados; o nome é No + número da linha
- Movimentos voltam para a linha do nó que eles movem, do mais interno ao mais externo
- Subárvore compartilhada sai uma vez e depois é referenciada pelo nome

```
T(1,0,0) · R(0,0,90) · Esfera(0.5)

No1; E; S 0.5; R 0,0,90; T 1,0,0
```